

Oo... adding a coherent character sequence to begin octal-literals

Proposal

Proposal to add `0o` and `0O` as an alternative (and preferred) sequence to introduce octal-literals.

The syntax rule to interpret integer literals starting with a zero as octal-literals probably dates back from the time people were still tinkering with 8 or 16 bit processors in their garages. This syntax rule is what might be called an “historical mistake”. Nowadays, this feature is hardly used and can be easily misunderstood or overseen by novice programmers leading to unexpected errors.

To allow future generations (of developers if not compilers) to correct this feature, I propose to add the character sequence `0o` as an alternative (and preferred) sequence to introduce an octal-literal. The prefix `0o` follows the model set by the prefix `0x` to introduce a hex-literal, and (since c++14) `0b` to introduce a binary-literal.

From <http://en.wikipedia.org/wiki/Octal> : *"Newer languages have been abandoning the prefix 0, as decimal numbers are often represented with leading zeroes. The prefix q was introduced to avoid the prefix o being mistaken for a zero, while the prefix 0o was introduced to avoid starting a numerical literal with an alphabetic character (like o or q), since these might cause the literal to be confused with a variable name. The prefix 0o also follows the model set by the prefix 0x used for hexadecimal literals in the C language; it is supported by Haskell,[11] OCaml,[12] Perl 6,[13] Python as of version 3.0,[14] Ruby,[15] Tcl as of version 9,[16] and it is intended to be supported by ECMAScript 6[17] (the prefix 0 has been discouraged in ECMAScript 3 and dropped in ECMAScript 5[18])."*

Examples

```
// The following literals all specify the same number.
```

```
int literal_octal_prefered      = 0o52;
int literal_octal_to_be_deprecated = 052;
int literal_decimal             = 42;
int literal_hex                 = 0x2A;
int literal_binary              = 0b00101010;
```

Effects on existing code

This proposal does not invalidate the existing syntax rule for integer literals starting with a zero. Also, under the current standard, any sequence starting with `0o` is illegal. As a consequence, the proposed modification will not break existing code.

Discussion

The objective of this proposal is to create the possibility, by adding this more coherent syntax to introduce octal-literals, to deprecate and phase out the current octal literal syntax in a future version (if so decided).

It is recommended that at the places where the `0o` sequence is introduced in the documentation, explicit attention is drawn to the existence of an incoherent (but backward compatible) syntax of integer literals starting with a zero.

It should be noted that a similar 'clean-up' is required in the specification of the [scanf](#) and [strtoul](#) families of C run-time library functions. The required modifications, although outside the scope of the C++ language, are also detailed below.

Technical Specification

Make the following edits (relative to [n4296](#)) with **insertions** and **removals** marked like so:

2.13.2 Integer literals [lex.icon]

```
octal-literal:
    0o octal-digit
    00 octal-digit
    0
    octal-literal 'opt octal-digit
```

1 An *integer literal* is a sequence of digits that has no period or exponent part, with optional separating single quotes that are ignored when determining its value. An integer literal may have a prefix that specifies its base and a suffix that specifies its type. The lexically first digit of the sequence of digits is the most significant. A *binary* integer literal (base two) begins with 0b or 0B and consists of a sequence of binary digits. An *octal* integer literal (base eight) begins with 0o, 0O or with the digit 0 and consists of a sequence of octal digits.²² A *decimal* integer literal (base ten) begins with a digit other than 0 and consists of a sequence of decimal digits. A *hexadecimal* integer literal (base sixteen) begins with 0x or 0X and consists of a sequence of hexadecimal digits, which include the decimal digits and the letters a through f and A through F with decimal values ten through fifteen. [*Example*: The number twelve can be written 12, 0o14, 014, 0XC, or 0b1100. The literals 1048576, 1'048'576, 0X100000, 0x10'0000, and 0o0'004'000'000 all have the same value. — *end example*]

A.2 Lexical conventions [gram.lex]

```
octal-literal:
    0o octal-digit
    00 octal-digit
    0
    octal-literal 'opt octal-digit
```

Addendum

While browsing the standard for the keyword 'octal', a second occurrence of octal encoded information was spotted in escape sequences of Character literals. The following modification could be considered in addition to the modification of octal-literals discussed above.

Note that the proposed modification hereunder also removes the restriction that limits the number of octal digits in an escape sequence to 3, as there is no such restriction for hexadecimal escape sequences. Of course, the specification that '*the value of a character literal is implementation-defined if it falls outside of the implementation-defined range defined for char ...*' is itself ill defined. Better would be to specify explicitly that the compiler produces a warning/error and/or to prescribe that the value equals the specified value modulus the implementation-defined range for char.

2.13.3 Character literals [lex.icon]

```
octal-escape-sequence:
    \o octal-digit
    \ octal-digit
    \ octal-digit octal-digit
    \ octal-digit octal-digit octal-digit
    octal-escape-sequence octal-digit
```

Table 6 — Escape sequences (2.13.3)

```
octal number nnn \onnn
octal number ooo \ooo
hex number hhh \xhhh
```

⁸ The escape \ooo consists of the backslash followed by one, two, or three octal digits that are taken to specify the value of the desired character. The escape \onnn consists of the backslash followed o followed by one or more octal digits that are taken to specify the value of the desired character. The escape \ooo consists of the backslash followed by one or more octal digits that are taken to specify the value of the desired character. The escape \xhhh consists of the backslash followed by x followed by one or more hexadecimal digits that are taken to specify the value of the desired character. There is no limit to the number of digits in an octal or a hexadecimal sequence. A sequence of octal or hexadecimal digits is terminated by the first character that is not an octal digit or a hexadecimal digit, respectively. The value of a character literal is implementation-defined if it falls outside of the implementation-defined range defined for char (for literals with no prefix) or wchar_t (for literals prefixed by L). [*Note*: If the value of a character literal prefixed by u, u8, or U is outside the range defined for its type, the program is ill-formed. — *end note*]

A.2 Lexical conventions [gram.lex]

```
octal-escape-sequence:
    \o octal-digit
```

~~octal-digit~~
~~octal-digit octal-digit~~
~~octal-digit octal-digit octal-digit~~
octal-escape-sequence octal-digit

Technical Specification C run time library functions: strtol, scanf

Although related, but outside the scope of the C++ language, proposed modifications to the technical specification of the C run time library functions `strtol` and `scanf` are given here for completeness:

strtol

Make the following edits (relative to [pubs.opengroup.org:strtol](https://pubs.opengroup.org/strtol)) with **insertions** and **removals** marked like so:

If the value of base is 0, the expected form of the subject sequence is that of a decimal constant, **binary constant**, octal constant, or hexadecimal constant, any of which may be preceded by a '+' or '-' sign. A decimal constant ***deprecated*** begins with a non-zero digit, and ***end deprecated*** consists of a sequence of decimal digits. **A binary constant consists of the prefix 0b or 0B followed by a sequence of the digits '0' or '1' only.** An octal constant consists of **the prefix 0o or 0O *deprecated* or** the prefix '0', optionally ***end deprecated*** followed by a sequence of the digits '0' to '7' only. A hexadecimal constant consists of the prefix 0x or 0X followed by a sequence of the decimal digits and letters 'a' (or 'A') to 'f' (or 'F') with values 10 to 15 respectively.

If the value of base is between 2 and 36, the expected form of the subject sequence is a sequence of letters and digits representing an integer with the radix specified by base, optionally preceded by a '+' or '-' sign. The letters from '0a' (or 'A') to 'z' (or 'Z') inclusive are ascribed the values 10 to 35; only letters whose ascribed values are less than that of base are permitted. If the value of base is **2, 8, 16**, the characters **0b or 0B, 0o or 0O, 0x or 0X *respectively*** may optionally precede the sequence of letters and digits, following the sign if present.

scanf

As the technical specification of `scanf` ([pubs.opengroup.org:scanf](https://pubs.opengroup.org/scanf)) is based on the specification of `strtol`, no modification is required to this specification. In practice, the documentation of `scanf` (in e.g. man pages) is often implicitly expressing the behaviour of the underlying `strtol` function. This documentation should be adapted accordingly. Example:

conversions

- i Matches an optionally signed integer; the next pointer must be a pointer to int.
The integer is read:
 - in base 16 if it begins with 0x or 0X,
 - in base 8 if it begins with **0o or 0O *deprecated* or 0 - *end deprecated***,
 - in base 2 if it begins with 0b or 0B**
 - and in base 10 otherwise.Only characters that correspond to the base are used.

Acknowledgements

Thanks to Axel Naumann for guidance, discussion and providing references to the standard documents.

The document style was borrowed from Doc. no. [N4340](#)